Three-Phase Phasor Extraction via DFT

Mathematical Derivation and MATLAB Implementation

claude

April 17, 2026

1 Problem Statement

Consider a balanced three-wire three-phase system whose line currents satisfy Kirchhoff's current law at every instant:

$$i_u(t) + i_v(t) + i_w(t) = 0 \qquad \forall\, t. \tag{1}$$

In the fundamental case each current is a sinusoid of amplitude $\hat{I}$, frequency f_0, and initial phase φ:

$$i_u(t) = \hat{I}\cos(2\pi f_0 t + \varphi), \tag{2}$$

$$i_v(t) = \hat{I}\cos\!\left(2\pi f_0 t + \varphi - \tfrac{2\pi}{3}\right), \tag{3}$$

$$i_w(t) = \hat{I}\cos\!\left(2\pi f_0 t + \varphi + \tfrac{2\pi}{3}\right). \tag{4}$$

The constraint (1) is automatically verified because $\cos\theta + \cos(\theta - 2\pi/3) + \cos(\theta + 2\pi/3) = 0$ for all θ.

The goal is to represent each current as a *phasor* $\underline{I}_x \in \mathbb{C}$ in the polar plane, and to extract those phasors numerically from sampled data using the Discrete Fourier Transform (DFT).

2 Phasor Representation

A real sinusoid $x(t) = \hat{X}\cos(2\pi f_0 t + \phi)$ is identified by its *complex phasor*

$$\underline{X} \;=\; \hat{X}\,\mathrm{e}^{\mathrm{j}\phi} \;=\; \hat{X}\angle\phi, \tag{5}$$

so that $x(t) = \mathrm{Re}\!\left[\underline{X}\,\mathrm{e}^{\mathrm{j}2\pi f_0 t}\right]$.

For the three-phase system above:

$$\underline{I}_u = \hat{I}\mathrm{e}^{\mathrm{j}\varphi}, \qquad \underline{I}_v = \hat{I}\mathrm{e}^{\mathrm{j}(\varphi - 2\pi/3)}, \qquad \underline{I}_w = \hat{I}\mathrm{e}^{\mathrm{j}(\varphi + 2\pi/3)}. \tag{6}$$

The three phasors are displaced by 120ř in the complex plane and satisfy $\underline{I}_u + \underline{I}_v + \underline{I}_w = 0$.

3 Discrete Fourier Transform – Brief Review

Let $x[n] = x(n/F_s)$, $n = 0, 1, \ldots, N-1$, be a uniformly sampled sequence with sampling frequency F_s and N total samples. The DFT is defined as

$$X[k] \;=\; \sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k n/N}, \qquad k = 0, 1, \ldots, N-1. \tag{7}$$

The DFT bin k corresponds to the physical frequency

$$f_k = \frac{k\,F_s}{N}. \tag{8}$$

For a pure cosine at f_0 sampled over an *integer number of periods* (i.e. $N = m\,F_s/f_0$ for some integer m), only two bins are non-zero: $k_1 = m$ (positive frequency) and k_{N-k_1} (negative frequency). The positive-frequency bin evaluates to

$$X[k_1] = \frac{N}{2}\,\hat{X}\,\mathrm{e}^{\mathrm{j}\phi}, \tag{9}$$

so the phasor can be recovered as

$$\boxed{\underline{X} = \frac{2}{N}\,X[k_1] = \frac{2}{N}\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}.} \tag{10}$$

Remark 1. *The factor $2/N$ accounts for the one-sided normalisation: the DFT at the positive-frequency bin captures half the total signal energy (the other half is in the conjugate bin at $N - k_1$).*

4 Derivation of the `phasor()` Function

Equation (10) is the theoretical backbone of the one-liner used in the MATLAB script:

$$\texttt{phasor(x)} \;\equiv\; \frac{2}{N}\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}, \tag{11}$$

where $k_1 = \texttt{round}(f_0 N/F_s)$ is the DFT bin index closest to the fundamental frequency f_0.

Step-by-step derivation

Step 1 – Represent the sinusoid as a sum of complex exponentials.

Using Euler's formula $\cos\theta = \frac{1}{2}(\mathrm{e}^{\mathrm{j}\theta} + \mathrm{e}^{-\mathrm{j}\theta})$, a single-tone signal of amplitude $\hat{X}$ and phase ϕ is

$$x[n] = \hat{X}\cos\!\left(\frac{2\pi k_1 n}{N} + \phi\right) = \frac{\hat{X}}{2}\,\mathrm{e}^{\mathrm{j}\phi}\,\mathrm{e}^{\mathrm{j}2\pi k_1 n/N} + \frac{\hat{X}}{2}\,\mathrm{e}^{-\mathrm{j}\phi}\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}. \tag{12}$$

Step 2 – Substitute into the DFT inner product at bin k_1.

Multiplying $x[n]$ by the DFT kernel $\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}$ and summing:

$$\begin{aligned}\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N} = \frac{\hat{X}}{2}\,\mathrm{e}^{\mathrm{j}\phi}\underbrace{\sum_{n=0}^{N-1}\mathrm{e}^{\mathrm{j}2\pi k_1 n/N}\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}}_{=N} \\ + \frac{\hat{X}}{2}\,\mathrm{e}^{-\mathrm{j}\phi}\underbrace{\sum_{n=0}^{N-1}\mathrm{e}^{-\mathrm{j}2\pi\cdot 2k_1 n/N}}_{=0\ \text{(orthogonality)}}.\end{aligned} \tag{13}$$

The second sum vanishes because $2k_1/N$ is not an integer multiple of $1/N$ (under the assumption $k_1 < N/2$), so the complex exponential completes full cycles that cancel. Hence

$$\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N} = \frac{N}{2}\,\hat{X}\,\mathrm{e}^{\mathrm{j}\phi}. \tag{14}$$

Step 3 – Normalise to obtain the phasor.

Multiplying both sides by $2/N$:

$$\frac{2}{N}\sum_{n=0}^{N-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N} = \hat{X}\,\mathrm{e}^{\mathrm{j}\phi} = \underline{X}. \qquad \square \tag{15}$$

This is exactly the complex phasor defined in (5), with modulus $|\underline{X}| = \hat{X}$ (peak value) and argument $\arg(\underline{X}) = \phi$.

Remark 2 (Validity for non-pure-tone signals)**.** *If the signal contains multiple harmonics or noise, the DFT at bin k_1 still isolates the fundamental component, because the DFT basis vectors are orthogonal. Contributions from other frequencies land on different bins and do not corrupt $X[k_1]$. This makes the method robust to distortion.*

5 MATLAB Implementation

The derivation translates verbatim into the following anonymous function:

```
% Number of samples in one fundamental period
N  = length(t);

% DFT bin index corresponding to f0
k1 = round(f0 * N / Fs);

% Anonymous function: single-bin DFT, normalised to peak phasor
phasor = @(x) (2/N) * sum( x .* exp(-1j * 2*pi * (0:N-1) * k1 / N) );

% Apply to each phase current
Iu = phasor(iu);    % Complex phasor of i_u(t)
Iv = phasor(iv);    % Complex phasor of i_v(t)
Iw = phasor(iw);    % Complex phasor of i_w(t)
```

Listing 1: Phasor extraction via single-bin DFT (MATLAB).

Line-by-line explanation

`N = length(t)` Total number of samples. Since `t` spans exactly one period $T = 1/f_0$ with step $1/F_s$, we have $N = F_s/f_0$, ensuring integer DFT bin alignment.

`k1 = round(f0*N/Fs)` Computes the fundamental bin index from (8): $k_1 = f_0 N/F_s$. The `round` guards against floating-point rounding when F_s/f_0 is not exactly integer.

`exp(-1j*2*pi*(0:N-1)*k1/N)` Evaluates the DFT kernel $\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}$ for all sample indices $n = 0, 1, \ldots, N-1$ in a single vectorised operation.

`x .* (...)` Element-wise multiplication of the signal vector by the kernel vector, implementing the inner product $\sum_n x[n]\,\mathrm{e}^{-\mathrm{j}2\pi k_1 n/N}$.

`sum(...)` Accumulates the inner product over all N samples.

`(2/N) * (...)` Applies the normalisation factor from (10), returning the complex phasor $\underline{X} = \hat{X}\mathrm{e}^{\mathrm{j}\phi}$.

6 Verification

For the balanced system (2)–(4) with $\hat{I} = 100\,\mathrm{A}$ and $\varphi = 0$:

$$\begin{aligned} \underline{I}_u &= 100\angle 0\check{\mathrm{r}}, \\ \underline{I}_v &= 100\angle -120\check{\mathrm{r}}, \\ \underline{I}_w &= 100\angle +120\check{\mathrm{r}}. \end{aligned}$$

These satisfy $|\underline{I}_u| = |\underline{I}_v| = |\underline{I}_w| = \hat{I}$ and $\underline{I}_u + \underline{I}_v + \underline{I}_w = 0$, consistent with the Kirchhoff constraint (1).

Property 1 (Symmetry check in MATLAB)**.** *The residual* `max(abs(iu+iv+iw))` *should be of order* 10^{-12} *A (double-precision machine epsilon times* $\hat{I}$*), confirming that the constraint is satisfied to numerical precision.*

7 Summary

Quantity	Symbol	MATLAB expression
Peak amplitude	$\hat{X}$	`abs(phasor(x))`
Phase angle	ϕ	`angle(phasor(x))`
Real part	$\hat{X}\cos\phi$	`real(phasor(x))`
Imaginary part	$\hat{X}\sin\phi$	`imag(phasor(x))`

The single-bin DFT approach is preferred over peak-detection or zero-crossing methods because it is robust to harmonics, noise, and non-integer periods, and because it generalises directly to unbalanced or distorted waveforms without any modification to the code.